



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: COMPILER DESIGN PROJECT EVALUATION (24CS2235F)

A.Y. 2026 – 2027

TITLE OF THE PROJECT	MiniLang: A Multi-Phase Compiler Implementation	
S. No.	University ID	Name
1	2420030638	Rayapureddi Rishi SriCharan
2	2420030161	Yennam Sesank Reddy
3	2420030662	G.Bala Nihith Reddy
4		
Name of the Guide		Dr. Krishna Kishore

Abstract

MiniLang is a compiler design project focused on designing and implementing a complete compiler for a small, C-inspired programming language called *MiniLang*. The language supports essential programming constructs such as integer and Boolean variables, arithmetic and relational expressions, assignments, conditional statements (if-else), loops (while), print statements, and optionally functions. The project demonstrates the complete process of converting a high-level source program into lower-level target code. MiniLang source code is processed through multiple compiler phases, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. The compiler is designed to provide meaningful error messages for lexical, syntax, and semantic errors, making it both an educational and practical implementation of compiler concepts.

The lexical analysis phase identifies keywords, identifiers, literals, operators, delimiters, and other tokens from the MiniLang source program using regular expressions and finite automata concepts. The parser then validates the token stream based on a context-free grammar designed for the language. A predictive recursive-descent parser or an LL(1)-based parser can be used after removing left recursion and applying left factoring to the grammar. During parsing, the compiler constructs an Abstract Syntax Tree (AST), which represents the logical structure of the program. A symbol table is maintained to store information about variables, their data types, declaration scopes, and usage. Semantic analysis is performed to identify errors such as undeclared variables, duplicate declarations, invalid assignments, and type mismatches.

After successful analysis, the compiler translates the AST into an intermediate representation using Three Address Code (TAC). The generated TAC supports expressions, assignments, Boolean expressions, branching, and flow-control statements through temporary variables and labels. An optimization module then improves the intermediate code by applying techniques such as constant folding, constant propagation, dead-code elimination, common subexpression elimination, basic-block optimization, DAG representation, and peephole optimization. Finally, the optimized intermediate code is converted into simplified assembly-like target code using a basic register allocation strategy. The final system can display the output of each compilation phase, including tokens, parse tree or AST, symbol table, three-address code, optimized code, and generated assembly, providing a clear demonstration of the complete compiler pipeline.

Signature of the Guide

HOD